

IA PARA DEVS

FASE 3 | AULA 03 -

RAG

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS.....	39
O QUE VOCÊ VIU NESTA AULA?	41
REFERÊNCIAS	42

O QUE VEM POR AÍ?

Nesta aula, exploraremos a **Retrieval-Augmented Generation (RAG)** integrada com **LangGraph**, criando uma arquitetura robusta que combina a recuperação de informações baseadas em similaridade semântica com a geração, dentro do contexto esperado, por meio de grafos de agentes.

Começaremos explicando como a RAG resolve algumas das limitações mais comuns com as LLMs tradicionais: conhecimento desatualizado, falta de informações específicas do domínio e tendência às famosas “alucinações”. Veremos como a integração com o LangGraph nos permite criar fluxos condicionais, onde a recuperação de documentos, validação de relevância e a geração de respostas ocorrem em nós especializados.

Em seguida, abordaremos os componentes técnicos que normalmente constituem uma arquitetura RAG: **embeddings models** de última geração, **vector stores** otimizados para busca semântica, **retrievers** inteligentes com re-ranking e **prompt engineering** avançado para contextualização. Implementaremos um sistema completo usando **StateGraph**, onde cada etapa do pipeline RAG é representada como um nó independente e testável.

Por fim, construiremos um exemplo prático usando o clássico livro “**Dom Casmurro**”, de Machado de Assis, demonstrando como implementar uma RAG com LangGraph para criar um assistente literário que pode responder perguntas complexas sobre a obra, demonstrando compreensão texto original e oferecendo rastreabilidade completa das fontes utilizadas.

HANDS ON

Nesta videoaula, serão apresentados os elementos fundamentais de uma aplicação no LangGraph. Abordaremos nós, arestas, estado compartilhado e ciclo de vida do grafo.

Na próxima seção, você poderá acompanhar o passo a passo realizado para a configuração do ambiente e dependências.



SAIBA MAIS

Configuração do ambiente e dependências

1. INSTALAÇÃO COMPLETA DE DEPENDÊNCIAS

```
# Instalar pacotes principais

pip install langgraph langchain langchain-openai
langchain-community

pip install faiss-cpu python-dotenv tiktoken

# Para processamento de texto avançado

pip install unstructured[local-inference]

# Criar requirements.txt

cat > requirements.txt << EOF

langgraph>=0.2.0

langchain>=0.3.0

langchain-openai>=0.2.0

langchain-community>=0.3.0

faiss-cpu>=1.8.0

python-dotenv>=1.0.0

tiktoken>=0.7.0

unstructured>=0.15.0

EOF
```

Código-fonte 1 – Instalação completa de dependências
Fonte: elaborado pelo autor (2025)

2. CONFIGURAÇÃO DE AMBIENTE

```
# Criar arquivo .env

cat > .env << EOF
```

```
OPENAI_API_KEY=sk-sua-chave-aqui

LANGCHAIN_TRACING_V2=true

LANGCHAIN_API_KEY=sua-chave-langsmith

EOF
```

Código-fonte 2 – Configuração de ambiente
Fonte: elaborado pelo autor (2025)

3. ESTRUTURA DO PROJETO

```
mkdir rag-langgraph-project

cd rag-langgraph-project

mkdir data src tests

touch src/__init__.py src/rag_graph.py src/states.py
src/nodes.py
```

Código-fonte 3 – Estrutura do projeto
Fonte: elaborado pelo autor (2025)

1. DOWNLOAD E PREPARAÇÃO DO TEXTO - “DOM CASMURRO”

```
import requests

import os

from pathlib import Path

def download_dom_casmurro():

    """Download Dom Casmurro do Project Gutenberg"""

    url = "https://www.gutenberg.org/files/55752/55752-0.txt"

    data_dir = Path("data")

    data_dir.mkdir(exist_ok=True)

    response = requests.get(url)
```

```
response.encoding = 'utf-8'

# Limpar metadados do Gutenberg
text = response.text

start_marker = "*** START OF THE PROJECT GUTENBERG
EBOOK"

end_marker = "*** END OF THE PROJECT GUTENBERG EBOOK"

start_idx = text.find(start_marker)
end_idx = text.find(end_marker)

if start_idx != -1:
    start_idx = text.find("\n", start_idx) + 1
else:
    start_idx = 0

if end_idx != -1:
    clean_text = text[start_idx:end_idx]
else:
    clean_text = text[start_idx:]

# Salvar texto limpo

file_path = data_dir / "dom_casmurro.txt"
with open(file_path, 'w', encoding='utf-8') as f:
    f.write(clean_text.strip())
```

```
print(f"Dom Casmurro salvo em: {file_path}")

print(f"Tamanho do texto: {len(clean_text)} caracteres")

    return file_path

# Executar download

text_file = download_dom_casmurro()
```

Código-fonte 4 – Download e preparação do texto “Dom Casmurro”
Fonte: elaborado pelo autor (2025)

IMPLEMENTAÇÃO RAG COM LANGGRAPH

1. Definindo o estado compartilhado

```
# src/states.py

from typing import TypedDict, List, Optional
from langchain_core.documents import Document

class RAGState(TypedDict):

    """Estado compartilhado do pipeline RAG"""

    question: str

    raw_documents: List[Document]

    filtered_documents: List[Document]

    context: str

    answer: str

    sources: List[str]

    confidence_score: float

    processing_steps: List[str]
```

Código-fonte 5 – Definindo o estado compartilhado
Fonte: elaborado pelo autor (2025)

2. Implementando os nós especializados

```
# src/nodes.py

import os

from typing import Dict, List

from langchain_community.document_loaders import TextLoader

from langchain_openai import OpenAIEmbeddings, ChatOpenAI

from langchain_community.vectorstores import FAISS

from langchain.text_splitter import RecursiveCharacterTextSplitter

from langchain_core.prompts import ChatPromptTemplate

from langchain_core.documents import Document

import logging

logger = logging.getLogger(__name__)

class RAGNodes:

    def __init__(self):

        self.embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

        self.llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.1)

        self.vectorstore = None

        self._setup_vectorstore()

    def _setup_vectorstore(self):

        """Inicializa o vector store com Dom Casmurro"""

        try:

            # Carregar documento
```

```
        loader = TextLoader("data/dom_casmurro.txt",
encoding="utf-8")

        documents = loader.load()

        # Chunking inteligente

        text_splitter = RecursiveCharacterTextSplitter(

            chunk_size=1000,

            chunk_overlap=200,

            length_function=len,

            separators=["\n\n", "\n", ". ", " ", ""]

        )

        split_docs = text_splitter.split_documents(documents)

        # Criar vector store

        self.vectorstore = FAISS.from_documents(split_docs, self.embeddings)

        logger.info(f"Vector store criado com {len(split_docs)} chunks")

    except Exception as e:

        logger.error(f"Erro ao configurar vector store: {e}")

        raise

    def retrieve_documents(self, state: Dict) -> Dict:

        """Nó de recuperação de documentos"""
```

```
question = state["question"]

try:

    # Busca semântica

    retriever = self.vectorstore.as_retriever(

        search_type="mmr", # Maximum Marginal
Relevance

        search_kwargs={

            "k": 6,

            "fetch_k": 20,

            "lambda_mult": 0.7

        }

    )

    raw_docs = retriever.invoke(question)

    steps = state.get("processing_steps", [])

    steps.append(f"Recuperados {len(raw_docs)} documentos
para: '{question[:50]}...'")

    return {

        "raw_documents": raw_docs,

        "processing_steps": steps

    }

except Exception as e:
```

```
logger.error(f"Erro na recuperação: {e}")

return {

    "raw_documents": [],

    "processing_steps": steps + [f"Erro na
recuperação: {str(e)}"]

}

def filter_and_rank(self, state: Dict) -> Dict:

    """Nó de filtragem e re-ranking de documentos"""

    question = state["question"]

    raw_docs = state["raw_documents"]

    if not raw_docs:

        return {

            "filtered_documents": [],

            "confidence_score": 0.0,

            "processing_steps":
state["processing_steps"] + ["Nenhum documento para filtrar"]

        }

    try:

        # Prompt para relevância

        relevance_prompt

        ChatPromptTemplate.from_template("""
```

Código-fonte 6 – Implementando os nós especializados
Fonte: elaborado pelo autor (2025)

Analise se o trecho é relevante para responder a pergunta.

Pergunta: {question}

```
Trecho: {content}

Responda apenas "SIM" se relevante ou "NÃO" se
irrelevante.

Se SIM, dê uma nota de 1-10 para a relevância.

Formato: SIM:8 ou NÃO
""")

filtered_docs = []
relevance_scores = []

for doc in raw_docs:
    response = self.llm.invoke(
        relevance_prompt.format_prompt(
            question=question,
            content=doc.page_content[:500]
        ).to_messages()
    )

    response_text = response.content.strip()

    if response_text.startswith("SIM"):
        try:
            score
            =
int(response_text.split(":")[1])
```

```
        filtered_docs.append(doc)

        relevance_scores.append(score)

    except:

        filtered_docs.append(doc)

    relevance_scores.append(5)

    # Ordenar por relevância

    if filtered_docs:

        sorted_pairs = sorted(zip(filtered_docs,
relevance_scores),
                                key=lambda x: x[1],
reverse=True)

        filtered_docs = [doc for doc, _ in
sorted_pairs[:4]]

        avg_confidence = sum(relevance_scores) /
len(relevance_scores) / 10

    else:

        avg_confidence = 0.0

    steps = state["processing_steps"]

    steps.append(f"Filtrados {len(filtered_docs)}
documentos relevantes")

    return {

        "filtered_documents": filtered_docs,

        "confidence_score": avg_confidence,

        "processing_steps": steps

    }
```

```
        except Exception as e:

            logger.error(f"Erro na filtragem: {e}")

        return {

            "filtered_documents": raw_docs[:3],      #
Fallback
            "confidence_score": 0.5,

            "processing_steps":
state["processing_steps"] + [f"Erro na filtragem: {str(e)}"]

        }

    def generate_context(self, state: Dict) -> Dict:

        """Nó de geração de contexto"""

        filtered_docs = state["filtered_documents"]

        if not filtered_docs:

            return {

                "context": "Nenhum contexto relevante encontrado.",

                "sources": [],

                "processing_steps":
state["processing_steps"] + ["Contexto vazio gerado"]

            }

        # Combinar documentos em contexto

        context_parts = []

        sources = []
```

```
        for i, doc in enumerate(filtered_docs, 1):
            context_parts.append(f"[Trecho {i}]\n{doc.page_content}")

            # Extrair fonte (primeiras palavras como
            # identificador)
            source_id = doc.page_content[:50].strip().replace('\n', '
')

            sources.append(f"Trecho {i}: {source_id}...")

        context = "\n\n".join(context_parts)

        steps = state["processing_steps"]

        steps.append(f"Contexto gerado com {len(context)}
        caracteres")

    return {

        "context": context,
        "sources": sources,
        "processing_steps": steps
    }

def generate_answer(self, state: Dict) -> Dict:
    """Nó de geração de resposta"""

    question = state["question"]

    context = state["context"]

    sources = state["sources"]
```

```
try:

    # Prompt para geração de resposta

    answer_prompt =
ChatPromptTemplate.from_template("""

    Você é um especialista em literatura brasileira,
    especificamente em Machado de Assis.
```

Código-fonte 7 – Análise do trecho relevante para responder a pergunta
Fonte: elaborado pelo autor (2025)

Responda a pergunta baseando-se **exclusivamente** no contexto fornecido.

```
CONTEXTO:

{context}

PERGUNTA: {question}

INSTRUÇÕES:

1. Use apenas informações do contexto
fornecido
2. Se não souber, diga que não há informação
suficiente
3. Cite trechos específicos quando relevante
4. Mantenha tom acadêmico mas acessível
5. Seja preciso e evite especulações

RESPOSTA:

""")

response = self.llm.invoke(
```

```
answer_prompt.format_prompt(
    question=question,
    context=context
).to_messages()
)

answer = response.content.strip()

# Adicionar referências às fontes
if sources:
    answer += "\n\n**Fontes consultadas:**\n"
for source in sources:
    answer += f"- {source}\n"

steps = state["processing_steps"]
steps.append("Resposta final gerada com sucesso")

return {
    "answer": answer,
    "processing_steps": steps
}

except Exception as e:
    logger.error(f"Erro na geração: {e}")
    return {
```

```
        "answer": f"Erro ao gerar resposta: {str(e)}",  
  
        "processing_steps":  
state["processing_steps"] + [f"Erro na geração: {str(e)}"]  
    }
```

Código-fonte 8 – Respondendo a pergunta baseando-se no contexto fornecido
Fonte: elaborado pelo autor (2025)

3. Construindo o grafo RAG

```
# src/rag_graph.py  
  
from langgraph.graph import StateGraph, END  
  
from src.states import RAGState  
  
from src.nodes import RAGNodes  
  
import logging  
  
logging.basicConfig(level=logging.INFO)  
  
logger = logging.getLogger(__name__)  
  
def create_rag_graph():  
    """Cria o grafo RAG com LangGraph"""  
  
    # Inicializar nós  
    rag_nodes = RAGNodes()  
  
    # Criar workflow  
    workflow = StateGraph(RAGState)  
  
    # Adicionar nós especializados  
    workflow.add_node("retrieve",  
rag_nodes.retrieve_documents)
```

```
        workflow.add_node("filter", rag_nodes.filter_and_rank)

        workflow.add_node("context",
rag_nodes.generate_context)

        workflow.add_node("generate",
rag_nodes.generate_answer)


    # Definir fluxo

    workflow.set_entry_point("retrieve")

    workflow.add_edge("retrieve", "filter")
    workflow.add_edge("filter", "context")
    workflow.add_edge("context", "generate")
    workflow.set_finish_point("generate")


# Compilar grafo

    app = workflow.compile()

    logger.info("Grafo RAG compilado com sucesso")
    return app

def run_rag_query(app, question: str) -> dict:

    """Executa uma consulta RAG"""

    initial_state = {

"question": question,

        "raw_documents": [],

        "filtered_documents": [],

        "context": "",

        "answer": "",
```

```
        "sources": [],
        "confidence_score": 0.0,
        "processing_steps": [f"Iniciando consulta:
{question}"]
    }

    result = app.invoke(initial_state)
    return result
```

Código-fonte 9 – Construindo o grafo RAG
Fonte: elaborado pelo autor (2025)

EXECUÇÃO E TESTES PRÁTICOS

1. Script principal de execução

```
# main.py

import os

from dotenv import load_dotenv

from src.rag_graph import create_rag_graph, run_rag_query

def main():
    # Carregar variáveis de ambiente
    load_dotenv()

    # Verificar API key
    if not os.getenv("OPENAI_API_KEY"):
        print("❌ OPENAI_API_KEY não encontrada no .env")
        return

    print("✅ Inicializando Sistema RAG com LangGraph...")
```

```
# Criar grafo

app = create_rag_graph()

print(" Sistema inicializado com sucesso!")

print("\n Base de conhecimento: Dom Casmurro (Machado
de Assis)")

print("=" * 50)

# Perguntas de teste

test_questions = [
    "Quem é Bentinho e o que ele conta no início da
história?",
    "Quem é Capitu e qual sua relação com Bentinho?",
    "O que significa o 'olhar oblíquo' de Capitu?",
    "Onde se passa a história?",
    "Qual é o conflito principal da narrativa?",
    "O narrador é confiável? Por quê?"
]

for i, question in enumerate(test_questions, 1):
    print(f"\n PERGUNTA {i}: {question}")
    print("-" * 70)

# Executar consulta

result = run_rag_query(app, question)
```

```
# Mostrar resultados

print(f" Confiança: {result['confidence_score']:.2f}")

print(f"          Documentos          recuperados:
{len(result['raw_documents'])}")

print(f"          Documentos          relevantes:
{len(result['filtered_documents'])}")

print(f"\n RESPOSTA:")

print(result['answer'])

# Mostrar processo (opcional)

if os.getenv("SHOW_PROCESS", "false").lower() ==
"true":

    print(f"\n PROCESSO:")

    for step in result['processing_steps']:

        print(f"    • {step}")

    print("\n" + "=" * 70)

# Modo interativo

print("\n  Modo  Interativo  (digite  'sair'  para
encerrar)")

while True:

    user_question = input("\n Sua pergunta: ").strip()

    if user_question.lower() in ['sair', 'exit',
'quit']:

        print(" Até logo!")
```

```
        break

    if not user_question:
        continue

    result = run_rag_query(app, user_question)
    print(f"\n RESPOSTA:")
    print(result['answer'])

if __name__ == "__main__":
    main()
```

Código-fonte 10 – Script principal de execução
Fonte: elaborado pelo autor (2025)

2. Visualização do Grafo

```
# visualize_graph.py

from src.rag_graph import create_rag_graph

def visualize_rag_graph():
    """Visualiza a estrutura do grafo RAG"""
    app = create_rag_graph()

    print(" Estrutura do Grafo RAG:")
    print("=" * 40)
    print(app.get_graph().draw_ascii())

    print("\n Descrição dos Nós:")
    print("-" * 40)
```

```
print("1. retrieve → Busca semântica de documentos")

print("2. filter → Filtragem e re-ranking por relevância")

print("3. context → Geração de contexto estruturado")

print("4. generate → Geração da resposta final")

if __name__ == "__main__":
    visualize_rag_graph()
```

Código-fonte 11 – Visualização do grafo
Fonte: elaborado pelo autor (2025)

ARQUITETURA RAG AVANÇADA COM LANGGRAPH

A implementação de RAG com LangGraph representa uma evolução significativa sobre pipelines lineares tradicionais que não utilizam a ferramenta. Enquanto sistemas RAG convencionais seguem uma sequência fixa de recuperação-geração, a arquitetura baseada em grafos permite fluxos condicionais, validação de qualidade em tempo real e adaptações inteligentes baseadas no contexto da consulta.

COMPONENTES TÉCNICOS AVANÇADOS

A qualidade do sistema RAG depende fundamentalmente da capacidade dos embeddings capturarem uma similaridade semântica precisa. Modelos modernos como `text-embedding-3-small` da OpenAI utilizam arquiteturas transformer otimizadas para capturar relações semânticas complexas, incluindo sinonímia (palavras diferentes que podem ser usadas num mesmo contexto), polissemia (quando uma palavra tem diferentes significados) e relações contextuais.

A implementação adequada requer consideração de múltiplos fatores: dimensionalidade dos vetores (o equilíbrio entre precisão e performance),

normalização para comparações consistentes e estratégias de chunking que preservem contexto semântico. Chunks muito pequenos perdem contexto, enquanto chunks grandes diluem informações específicas.

Vector stores e otimização de busca

Em sistemas RAG, **vector stores** são bancos de dados especializados em armazenar e recuperar **representações vetoriais** de textos (embeddings). Em vez de buscar por palavras-chave, como num buscador tradicional, o sistema compara vetores em um espaço multidimensional para encontrar passagens semanticamente parecidas com a consulta do usuário.

Isso é importante porque um bom sistema RAG depende de encontrar não só textos **com as mesmas palavras**, mas textos **com o mesmo sentido**. A precisão na recuperação de contexto relevante é o que diferencia uma resposta rasa de uma resposta útil e informada.

Ferramentas como FAISS

A **FAISS (Facebook AI Similarity Search)** é uma das bibliotecas mais usadas para implementar esse tipo de busca vetorial. Ela permite realizar buscas ultrarrápidas mesmo quando **hâmilhões de vetores** armazenados, graças a algoritmos especializados como:

- **IVF (Inverted File Index):** divide os vetores em clusters e busca apenas nos mais relevantes, reduzindo o tempo computacional;
- **HNSW (Hierarchical Navigable Small World):** constrói uma estrutura de grafo eficiente para navegar rapidamente entre os vetores mais parecidos.

Boas práticas de configuração

Para que o vector store funcione bem, é preciso ajustar:

- **Tipo de índice:** por exemplo, usar HNSW para consultas com latência baixa ou IVF para datasets maiores com mais controle de precisão;
- **Quantização:** reduz a precisão dos vetores para economizar memória e acelerar a busca (útil quando se tem bilhões de vetores);

- **Clustering:** pré-agrupa vetores similares, agilizando o processo de seleção durante a busca.

Relevância vs. Diversidade (MMR)

Uma técnica adicional importante é o **MMR (Maximum Marginal Relevance)**. Em vez de trazer apenas os resultados similares ao vetor de consulta, o MMR tenta equilibrar relevância com diversidade, evitando repetições. Isso é especialmente útil em RAG quando queremos que os trechos recuperados cubram diferentes aspectos da pergunta do usuário.

Estratégias de Retrieval Inteligente

Sistemas RAG de última geração não usam uma única abordagem para encontrar trechos relevantes no corpus de documentos. Eles combinam **diferentes estratégias de retrieval**, cada uma com pontos fortes para diferentes tipos de pergunta. Veja como isso funciona:

Dense Retrieval

Essa é a forma mais comum em sistemas RAG modernos. Aqui, a busca é feita com **vetores gerados por modelos de embeddings**. Tanto a consulta do usuário quanto os documentos são transformados em vetores de alta dimensionalidade e o sistema recupera os trechos mais “próximos” no espaço vetorial.

- **Força:** alta capacidade de entender semelhança de significado, mesmo que não haja palavras idênticas;
- **Exemplo:** se o usuário perguntar "como funciona o sono REM?", o sistema pode trazer um trecho que fala de "fase dos sonhos profundos", mesmo sem citar a palavra “REM”.

Sparse Retrieval (ex: BM25)

Essa estratégia é baseada em **palavras-chave** e frequência de termos (como nos buscadores tradicionais). O algoritmo BM25 calcula uma pontuação para cada documento com base na presença (ou ausência) das palavras da consulta.

- **Força:** muito bom quando o usuário usa termos técnicos ou exatos que devem aparecer no texto;
- **Exemplo:** quando a pergunta é “Quais as leis da termodinâmica?”, o sistema vai procurar documentos que mencionam essas palavras exatamente.

Hybrid Retrieval

Aqui ocorre a **combinação inteligente de Dense e Sparse Retrieval**. O sistema pondera os dois resultados com pesos ajustáveis, levando em conta o tipo de consulta. Em alguns casos, a parte densa (semântica) pode pesar mais e, em outros, a parte esparsa (palavras-chave).

- **Força:** aproveita o melhor das duas abordagens;
- **Exemplo:** perguntas vagas ou ambíguas podem se beneficiar da semântica; perguntas técnicas funcionam melhor com keywords.

Multi-Vector Retrieval

Em vez de usar um único vetor para representar cada documento, o sistema cria **múltiplos vetores por trecho**, cada um representando uma faceta semântica diferente (por exemplo: definição, exemplo e consequência).

- **Força:** permite capturar nuances complexas de significado e recuperar documentos que respondem a diferentes dimensões da pergunta;
- **Exemplo:** para a pergunta “quais são os riscos da IA?”, o sistema pode trazer trechos que explicam os riscos técnicos, sociais e regulatórios — cada um mapeado por um vetor diferente.

IMPLEMENTAÇÃO DE NÓS ESPECIALIZADOS

Nó de recuperação adaptativa

```
def adaptive_retrieval(state: dict) -> dict:

    """Retrieval que adapta estratégia baseada no tipo de
    pergunta"""

    question = state["question"]

    question_type = classify_question_type(question)

    if question_type == "factual":

        # Priorizar precisão sobre recall

        search_kwargs = {"k": 3, "score_threshold": 0.8}
    elif question_type == "analytical":

        # Mais documentos para análise abrangente

        search_kwargs = {"k": 8, "fetch_k": 30}
    else:

        # Configuração balanceada

        search_kwargs = {"k": 5, "fetch_k": 20}

    retriever = vectorstore.as_retriever(search_kwargs=search_kwargs)

    documents = retriever.invoke(question)

    return {"raw_documents": documents}
```

Código-fonte 12 – Nó de recuperação adaptativa
Fonte: elaborado pelo autor (2025)

Nó de validação de qualidade

```
def quality_validation(state: dict) -> dict:

    """Valida qualidade dos documentos recuperados"""

    documents = state["filtered_documents"]

    question = state["question"]

    quality_scores = []

    for doc in documents:

        # Validações múltiplas

        relevance_score = calculate_semantic_relevance(question,
doc)

        completeness_score = assess_information_completeness(question, doc)

        coherence_score = evaluate_text_coherence(doc)

        composite_score = (

            relevance_score * 0.5 +

            completeness_score * 0.3 +

            coherence_score * 0.2

        )

        quality_scores.append(composite_score)

    # Filtrar apenas documentos de alta qualidade

    high_quality_docs = [
```

```
        doc    for doc,    score    in    zip(documents,
quality_scores)

        if score > 0.7

    ]

    return {

        "filtered_documents": high_quality_docs,

        "quality_scores": quality_scores

    }
```

Código-fonte 13 – Nó de validação de qualidade
Fonte: elaborado pelo autor (2025)

PADRÕES AVANÇADOS DE IMPLEMENTAÇÃO

Roteamento condicional baseado em confiança

```
def confidence_router(state: dict) -> str:

    """Roteia fluxo baseado na confiança dos resultados"""

    confidence = state["confidence_score"]

    if confidence > 0.8:

        return "generate_answer"

    elif confidence > 0.5:

        return "expand_search"

    else:

        return "fallback_strategy"

workflow.add_conditional_edges(

    "filter",

    confidence_router,
```

```
{  
    "generate_answer": "generate",  
    "expand_search": "expanded_retrieval",  
    "fallback_strategy": "fallback"  
}  
)
```

Código-fonte 14 – Roteamento condicional baseado em confiança
Fonte: elaborado pelo autor (2025)

Cache inteligente de embeddings

```
class SmartEmbeddingCache:  
    def __init__(self, max_size=10000):  
        self.cache = {}  
        self.access_count = {}  
        self.max_size = max_size  
  
    def get_embedding(self, text: str):  
        text_hash = hashlib.md5(text.encode()).hexdigest()  
  
        if text_hash in self.cache:  
            self.access_count[text_hash] += 1  
            return self.cache[text_hash]  
  
        # Gerar novo embedding  
        embedding = embedding_model.embed_query(text)
```

```
# Gerenciar tamanho do cache

if len(self.cache) >= self.max_size:

    self._evict_least_used()

self.cache[text_hash] = embedding

self.access_count[text_hash] = 1

return embedding
```

Código-fonte 15 – Cache inteligente de embeddings
Fonte: elaborado pelo autor (2025)

OTIMIZAÇÃO DE PERFORMANCE

Chunking semanticamente inteligente

```
def semantic_chunking(text: str, max_chunk_size: int = 1000) -> List[str]:

    """Chunking que preserva unidades semânticas"""

    sentences = split_into_sentences(text)

    chunks = []

    current_chunk = ""

    for sentence in sentences:

        potential_chunk = current_chunk + " " + sentence

        if len(potential_chunk) > max_chunk_size:

            if current_chunk:

                chunks.append(current_chunk.strip())

            current_chunk = sentence

    current_chunk = sentence
```

```
        else:

            # Verificar coerência semântica

            if is_semantically_coherent(current_chunk, sentence):

                current_chunk = potential_chunk

            else:

                chunks.append(current_chunk.strip())

                current_chunk = sentence

        if current_chunk:

            chunks.append(current_chunk.strip())

    return chunks
```

Código-fonte 16 – Chunking semanticamente inteligente
Fonte: elaborado pelo autor (2025)

Parallel Processing de nós independentes

```
import asyncio

from concurrent.futures import ThreadPoolExecutor

async def parallel_document_processing(documents:
List[Document]) -> List[Dict]:

    """Processa documentos em paralelo para melhor
    performance"""

    async def process_single_doc(doc):

        # Simulação de processamento pesado

        relevance = await calculate_relevance_async(doc)

        sentiment = await analyze_sentiment_async(doc)

        entities = await extract_entities_async(doc)
```

```
        return {  
            "document": doc,  
            "relevance": relevance,  
            "sentiment": sentiment,  
            "entities": entities  
        }  
  
    # Processar em paralelo  
    tasks = [process_single_doc(doc) for doc in documents]  
    results = await asyncio.gather(*tasks)  
  
    return results
```

Código-fonte 17 – Parallel processing de nós independentes
Fonte: elaborado pelo autor (2025)

AVALIAÇÃO E MÉTRICAS DE SISTEMAS RAG

Métricas quantitativas

```
class RAGEvaluator:  
    def __init__(self):  
        self.metrics = {}  
  
    def evaluate_retrieval(self, retrieved_docs,  
        relevant_docs):  
        """Avalia qualidade da recuperação"""  
        retrieved_ids = {doc.metadata.get('id') for doc in  
            retrieved_docs}  
        relevant_ids = {doc.metadata.get('id') for doc in  
            relevant_docs}
```

```
# Precision: % de documentos recuperados que são
relevantes

precision = len(retrieved_ids & relevant_ids) /
len(retrieved_ids)

# Recall: % de documentos relevantes que foram recuperados

recall = len(retrieved_ids & relevant_ids) /
len(relevant_ids)

# F1 Score: harmônica entre precision e recall

f1 = 2 * (precision * recall) / (precision +
recall) if (precision + recall) > 0 else 0

return {"precision": precision, "recall": recall,
"f1": f1}

def evaluate_generation(self, generated_answer,
reference_answer):
    """Avalia qualidade da geração"""
    from rouge_score import rouge_scorer

    scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2',
'rougeL'])

    scores = scorer.score(reference_answer,
generated_answer)

    # Adicionar métricas customizadas

    factual_accuracy =
self.check_factual_accuracy(generated_answer,
reference_answer)
```

```
coherence_score =
self.assess_coherence(generated_answer)

return {
    "rouge_scores": scores,
    "factual_accuracy": factual_accuracy,
    "coherence": coherence_score
}
```

Código-fonte 18 – Métricas Quantitativas
Fonte: elaborado pelo autor (2025)

Monitoramento em produção

```
class RAGMonitor:

    def __init__(self):
        self.query_logs = []
        self.performance_metrics = {}

    def log_query(self, query_data):
        """Log detalhado de cada consulta"""
        log_entry = {
            "timestamp": datetime.now(),
            "question": query_data["question"],
            "retrieval_time":
query_data["retrieval_time"],
            "generation_time":
query_data["generation_time"],
            "confidence_score":
query_data["confidence_score"],
```

```
        "num_documents_retrieved":  
len(query_data["documents"]),  
  
        "user_feedback":      None          #      Preenchido  
posteriormente  
  
    }  
  
    self.query_logs.append(log_entry)  
  
    # Detectar padrões problemáticos  
    self.detect_anomalies(log_entry)  
  
    def detect_anomalies(self, log_entry):  
        """Detecção de anomalias em tempo real"""  
        if log_entry["confidence_score"] < 0.3:  
            self.alert("Low confidence query detected")  
  
            if log_entry["retrieval_time"] > 5.0:  
                self.alert("Slow retrieval detected")  
  
            if log_entry["num_documents_retrieved"] == 0:  
                self.alert("No documents retrieved")
```

Código-fonte 19 – Monitoramento em Produção
Fonte: elaborado pelo autor (2025)

MERCADO, CASES E TENDÊNCIAS

REVOLUÇÃO RAG EM ESCALA EMPRESARIAL

O mercado de soluções RAG tem experimentado crescimento explosivo, com investimentos globais superando \$15 bilhões em 2024. Empresas líderes reconhecem RAG como tecnologia fundamental para democratizar acesso a conhecimento organizacional e reduzir custos operacionais por meio de automação inteligente.

Meta AI - RAG Foundation Research e Implementação

A Meta, por meio do Facebook AI Research (FAIR), não apenas cocriou a arquitetura RAG original, mas continua liderando inovações na área. Seu sistema interno de RAG processa mais de 100 milhões de consultas diárias para suporte ao desenvolvimento, moderação de conteúdo e pesquisa científica. A implementação utiliza índices distribuídos com mais de 50 bilhões de documentos, demonstrando escalabilidade sem precedentes.

O impacto quantificado inclui redução de 40% no tempo de resolução de tickets de suporte técnico, melhoria de 60% na precisão de respostas automatizadas e economia anual de \$200 milhões em custos operacionais. A Meta abriu parte desta tecnologia como open-source, acelerando a adoção global.

Samsung SDS - SKE-GPT: Kubernetes Troubleshooting Inteligente

A Samsung SDS revolucionou as operações DevOps com SKE-GPT, sistema RAG especializado no diagnóstico automatizado de clusters Kubernetes. O sistema integra logs de sistema, documentação técnica, bases de conhecimento de incidentes históricos e melhores práticas operacionais em um corpus unificado de mais de 10 milhões de documentos técnicos.

A arquitetura utiliza embeddings especializados treinados em linguagem técnica de infraestrutura, vector stores otimizados para consultas temporais e agentes especializados em diferentes componentes do ecossistema Kubernetes. Resultados demonstram uma redução de 75% no tempo médio de resolução de

incidentes, diminuição de 50% em escalações para especialistas seniores e aumento de 85% na taxa de autorresolução de problemas. Leia mais nos dois artigos a seguir:

[“Case Study: Samsung SDS Uses Retrieval Augmented Generation for Kubernetes Troubleshooting”](#)

[“Samsung SDS Leverages Retrieval Augmented Generation for Kubernetes Troubleshooting”](#)

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos a implementação avançada de RAG utilizando LangGraph, estabelecendo uma base sólida para sistemas de IA que combinam recuperação inteligente de informações com geração contextualizada. Começamos com a configuração completa do ambiente de desenvolvimento, incluindo dependências atualizadas, estrutura de projeto profissional e preparação adequada de dados usando a obra “Dom Casmurro”, do escritor Machado de Assis, como corpus de conhecimento. Aprendemos a baixar, limpar e processar textos de forma reproduzível.

Implementamos uma arquitetura RAG completa usando StateGraph, onde cada etapa do pipeline é representada por nós especializados: recuperação de documentos com busca semântica otimizada, filtragem e re-ranking baseados em relevância, geração de contexto estruturado e produção de respostas com rastreabilidade completa das fontes. Exploramos técnicas avançadas, incluindo Maximum Marginal Relevance, para diversidade de resultados, estratégias de chunking semanticamente inteligente, sistemas de cache para otimização de performance e padrões de roteamento condicional baseados em confiança dos resultados.

Analizamos casos reais de implementação em escala empresarial, desde Meta AI, com processamento de bilhões de documentos, até Samsung SDS, automatizando troubleshooting de Kubernetes, demonstrando ROI quantificável e impacto transformacional em operações organizacionais. Os conceitos expostos e esse “stack” tecnológico nos prepara para a implementação de sistemas RAG robustos em produção, integrando recuperação de informações com modelos de linguagem de forma escalável e otimizada para casos de uso específicos, explorando as funcionalidades que temos disponíveis com a ferramenta LangGraph.

REFERÊNCIAS

CAI, K. **Contextual AI raises \$80 million for enhanced RAG platform**. Reuters. 2024. Disponível em: <https://www.reuters.com/technology/artificial-intelligence/contextual-ai-raises-80-mln-model-enhancing-technique-2024-08-01/>. Acesso em: 01 ago. 2025.

CONTEXTUAL AI. **Introducing RAG 2.0**. 2024. Disponível em: <https://contextual.ai/introducing-rag2/>. Acesso em: 01 ago. 2025.

GUTENBERG PROJECT. **Dom Casmurro – Machado de Assis**. [s.d.]. Disponível em: <https://www.gutenberg.org/ebooks/55752>. Acesso em: 01 ago. 2025.

LANGCHAIN Authors. **API Reference and Configuration Guide**. [s.d.]. Disponível em: <https://docs.langchain.com/docs/guides>. Acesso em: 01 ago. 2025.

LANGCHAIN Authors. **LangChain Documentation**. [s.d.]. Disponível em: <https://docs.langchain.com>. Acesso em: 01 ago. 2025.

LANGGRAPH Authors. **LangGraph Official GitHub Repository**. [s.d.]. Disponível em: <https://github.com/langchain-ai/langgraph>. Acesso em: 01 ago. 2025.

SHEBBAR. **Samsung SDS leverages Retrieval-Augmented Generation for Kubernetes Troubleshooting**. Medium. 2024. Disponível em: <https://medium.com/@srini.hebbar/samsung-sds-leverages-retrieval-augmented-generation-for-kubernetes-troubleshooting-b134fc282a68>. Acesso em: 01 ago. 2025.

PALAVRAS-CHAVE

Retrieval Augmented Generation. LangGraph. LangChain.

EMEND



POS TECH